

MAS Pre-Test — Answer Key & Explanations

Agentic AI Coding Fitness | Week 9 | 50 Questions with Explanations

Quick Answer Reference

Circle answers for rapid grading. Explanations follow on the next pages.

Q#	Answer	Q#	Answer	Q#	Answer	Q#	Answer	Q#	Answer
Q1	B	Q11	C	Q21	C	Q31	C	Q41	B
Q2	C	Q12	C	Q22	C	Q32	B	Q42	C
Q3	D	Q13	B	Q23	C	Q33	C	Q43	C
Q4	B	Q14	B	Q24	B	Q34	C	Q44	B
Q5	C	Q15	B	Q25	B	Q35	B	Q45	B
Q6	B	Q16	B	Q26	B	Q36	B	Q46	B
Q7	C	Q17	B	Q27	C	Q37	C	Q47	B
Q8	B	Q18	C	Q28	C	Q38	D	Q48	B
Q9	B	Q19	C	Q29	B	Q39	C	Q49	C
Q10	B	Q20	B	Q30	B	Q40	B	Q50	C

Score Interpretation

Score	Band	Interpretation	Workshop focus
45-50	Expert	Strong MAS foundation. Focus on advanced implementation patterns.	Exercises 10-14
35-44	Advanced	Good understanding. Gaps likely in communication protocols or ADK.	Exercises 7-14
25-34	Intermediate	Solid basics. Need to strengthen agent design and FIPA-ACL.	Full workshop
15-24	Beginner	Core MAS concepts are forming. Attend all sessions actively.	Full workshop + exercises 1-7
0-14	Foundational	New to MAS. Great opportunity — this workshop will be transformative.	Full workshop + extra reading

Q1 What is the most accurate definition of a Multi-Agent System (MAS)?

- A A single AI model that processes multiple input prompts in parallel
- B A system of multiple interacting autonomous agents, each with its own goals, perceptions, and capabilities**
- C A distributed database where multiple servers handle different tables
- D A neural network with multiple hidden layers and attention heads

Explanation: A MAS consists of multiple autonomous agents that perceive their environment, make decisions, and interact with each other to achieve individual or collective goals. Each agent is distinct.

Q2 Which of the following is NOT a benefit of using a Multi-Agent System compared to a single agent?

- A Modularity — each agent handles a focused concern
- B Specialization — agents can have domain-specific tools and knowledge
- C Lower LLM token cost — fewer total API calls needed**
- D Fault tolerance — if one agent fails, others continue working

Explanation: MAS typically increases token usage and coordination overhead, not decreases it. The real benefits are modularity, specialization, parallelism, and fault tolerance.

Q3 According to Wooldridge's agent theory, which property describes an agent that acts on behalf of users without constant human direction?

- A Reactivity — responding to environmental changes
- B Proactivity — taking initiative to achieve goals
- C Social ability — communicating with other agents
- D Autonomy — making decisions independently**

Explanation: Autonomy means agents operate without step-by-step human control, making independent decisions. Reactivity is responding to stimuli. Proactivity is goal-directed initiative. Social ability is communication.

Q4 In MAS theory, "emergence" refers to:

- A A new agent type that replaces all other specialized agents
- B Complex system-level behavior arising from interactions between agents, not explicitly programmed into any single agent**
- C The process of an LLM generating new agent code automatically
- D An error state when two agents simultaneously request the same resource

Explanation: Emergence is when the collective behavior of interacting agents produces outcomes that none of the individual agents were programmed to produce alone. Ant colonies are a classic example.

Q5 Which real-world system is the BEST analogy for a cooperative Multi-Agent System?

- A A single surgeon performing an operation alone
- B A calculator executing one formula at a time
- C A hospital where doctors, nurses, pharmacists, and specialists collaborate on patient care**
- D A spreadsheet with multiple formulas in separate cells

Explanation: A hospital is the classic analogy: each department (agent) has specialized expertise and tools, they communicate and coordinate, and collectively they deliver outcomes no single person could manage alone.

Q6 What is "coordination overhead" in a Multi-Agent System?

- A The electricity cost of running multiple GPU servers
- B The extra communication, synchronization, and conflict-resolution work required between agents**
- C The number of lines of code needed to define each agent
- D The delay introduced by rate limiting on external APIs

Explanation: Coordination overhead is the cost of keeping agents aligned: message passing, shared state updates, conflict resolution, and consensus. This is why you should start with the minimum number of agents needed.

Q7 When should you prefer a single agent over a Multi-Agent System?

- A When the task requires real-time parallel data collection from multiple sources
- B When the task needs specialized expertise across three distinct domains
- C When a simple sequential task comfortably fits within one agent's context and capabilities**
- D When the system must continue working if one component fails

Explanation: If a task is simple, sequential, and fits within one agent's capability, adding multiple agents just adds coordination overhead with no benefit. Challenge: always ask "do I really need MAS here?"

Q8 An "agent" in MAS is best described as an entity that:

- A Executes one fixed function and returns a result
- B Perceives its environment, makes decisions, and takes actions to achieve its goals**
- C Stores data in a structured format for other systems to query
- D Generates images or text from user prompts

Explanation: The core agent loop is: Perceive (sense environment) -> Decide (reason about state) -> Act (execute action) -> Repeat. This perception-decision-action cycle is what distinguishes an agent from a simple function.

Q9	What is the minimum number of interacting entities needed to technically form a "Multi-Agent System"?
A	1
B	2
C	5
D	10
Explanation:	A Multi-Agent System requires at least 2 agents that interact. "Multi" means more than one. However, meaningful MAS typically have 3+ agents with distinct roles.

Q10	Which scenario BEST demonstrates why MAS is needed over a single agent?
A	A chatbot answering simple FAQ questions from a knowledge base
B	A system that simultaneously monitors energy in 50 hotel zones, each requiring different control logic and local sensor access
C	A tool that converts PDF documents to text format
D	An AI that generates captions for social media images
Explanation:	50 zones each needing local sensing, domain logic, and control — this requires parallelism, specialization, and scale that justifies a MAS. The other options are straightforward single-agent tasks.

Q1
1**Before designing a MAS, the FIRST question you should answer is:**

- A Which cloud provider to deploy on
- B Which programming language and LLM framework to use

C What specific problem you want to solve and what measurable success looks like

- D How many API tokens your budget allows per month

Explanation: Without a clear problem statement and success criteria, all other design decisions are premature. The problem statement answers: who is affected, what is failing, and what does "good" look like?

Q1
2**You need to simultaneously fetch weather data, stock prices, and news headlines. Which reason BEST justifies using a MAS?**

- A It is cheaper to run three small agents than one large agent
- B Single agents cannot make HTTP requests
- C Multiple independent parallel workstreams benefit from concurrent execution**
- D The data sources require three different programming languages

Explanation: Parallel data fetching is one of the strongest justifications for MAS. Three independent data sources can be fetched simultaneously by three agents, reducing total latency significantly.

Q1
3**When defining each agent (Design Question 4), which four properties are most important to specify?**

- A Name, cloud region, deployment date, monthly cost
- B Goal, Backstory/Persona, Tools it can call, Output it produces**
- C Framework, LLM model, temperature setting, max tokens
- D Color code, icon, font size, display position in UI

Explanation: Each agent needs: a clear Goal (what it achieves), Backstory/Persona (its expertise/role), Tools (what it can call/execute), and Output (what data it produces for the next step or the user).

Q1
4**A fraud detection agent only sees transaction patterns but cannot observe the customer's actual intent or location. This environment is:**

- A Fully observable — the agent sees all transaction data
- B Partially observable — key state variables (intent, location) are hidden**
- C Static — the environment does not change during analysis
- D Deterministic — the same transaction always produces the same risk score

Explanation: Partial observability occurs when an agent cannot access all relevant state information. In fraud detection, intent is hidden, requiring the agent to estimate risk from indirect signals.

**Q1
5**

When is "state estimation" needed in a MAS design?

A When agents have too many tools and need to choose between them

B When the environment is partially observable and agents must infer hidden state from indirect signals

C When the LLM model does not know the current date

D When two agents produce conflicting outputs

Explanation: State estimation (e.g., Bayesian filtering, prediction models) is needed when an agent cannot directly observe full system state. It must infer hidden variables from observable signals.

**Q1
6**

Design Question 6 asks about agent collaboration mode. In a market-based MAS, multiple delivery agents submit cost bids for a logistics contract. This demonstrates which mode?

A Cooperative — all delivery agents share the same goal

B Competitive — agents bid against each other for the contract

C Adversarial — agents actively try to sabotage each other

D Hierarchical — a central agent assigns tasks without bidding

Explanation: Market-based or auction mechanisms are the classic form of competitive MAS. Agents self-interestedly bid for tasks, and the best bid wins. This is the Contract Net Protocol in action.

**Q1
7**

According to the design principle, which question should you ask yourself after listing all your planned agents?

A Which agent should have the highest LLM temperature?

B Could any two agents be merged without violating separation of concerns?

C Which agent should run first alphabetically?

D How many tokens will each agent use per day?

Explanation: The key design discipline is minimizing agents. For every pair, ask: "Could we merge these without hurting modularity?" More agents = more coordination overhead. Each agent must justify its existence.

**Q1
8**

Design Question 8 asks how MAS fits your architecture. What should a production MAS expose to a frontend application?

A Direct access to agent Python objects via import statements

B Raw LLM prompt/response logs

C A well-defined API surface such as REST, WebSocket, or event stream

D Database connection strings for each agent's memory store

Explanation: MAS should behave as a backend service with a clean API boundary. The frontend triggers workflows via REST/WebSocket and receives results via callbacks, streams, or polling — not by directly instantiating agents.

Q1 9	A developer says "I'll build a MAS with 15 agents because each sensor type needs its own agent." What is the most important concern to raise?
A	Whether 15 is an even number
B	Whether the cloud budget supports 15 agents
C	Whether 15 agents can be merged into fewer without losing necessary specialization — more agents means more coordination overhead
D	Whether all 15 agents should use the same LLM model
Explanation:	Starting with the minimum number of agents is a key design principle. Ask whether multiple sensor agents could be consolidated into a single SensorMonitorAgent with a tool for each sensor type.
Q2 0	You are designing a food safety monitoring MAS. The system must both continuously monitor temperatures AND generate regulatory reports. What does this suggest about agent design?
A	One MonitoringAgent handles both — less coordination overhead
B	Two separate agents: TempMonitorAgent (continuous loop) and ReportAgent (triggered on event) — different execution patterns require different agent types
C	Monitoring should be done manually; only reporting needs an agent
D	Both tasks should be combined into a single LLM prompt with two instructions
Explanation:	Continuous monitoring (LoopAgent pattern) and event-triggered report generation (SequentialAgent pattern) have fundamentally different execution models, justifying two separate agents.

Q2 1 In Google ADK, which agent type executes sub-agents one after another in a fixed sequence, passing shared state between them?

- A ParallelAgent
- B LoopAgent
- C SequentialAgent**
- D CoordinatorAgent

Explanation: SequentialAgent runs sub_agents in the order listed. Each agent writes to session.state and the next agent reads from it. Output of one step feeds the next.

Q2 2 Which ADK agent type is best for simultaneously fetching weather data from three different APIs to minimize total latency?

- A SequentialAgent — runs all fetches in order
- B LoopAgent — repeats until all fetches succeed
- C ParallelAgent — executes all sub-agents concurrently**
- D LlmAgent — uses LLM to decide which API to call

Explanation: ParallelAgent runs all sub-agents concurrently. For independent data fetches with no dependencies, this minimizes total wall-clock time from $N * \text{latency}$ to $\max(\text{latency})$.

Q2 3 A LoopAgent in Google ADK stops executing when:

- A The first sub-agent raises a Python exception
- B All agents produce identical outputs
- C max_iterations is reached OR any sub-agent returns an event with escalate=True**
- D The LLM model rate limit is exceeded

Explanation: LoopAgent has two termination conditions: hitting the max_iterations limit, or a sub-agent explicitly signaling completion by returning escalate=True in its event actions.

Q2 4 In Google ADK, how does an LLM Agent dynamically route a task to a different specialist agent?

- A By writing the agent name to a shared file and polling
- B By calling transfer_to_agent(agent_name="TargetAgent")**
- C By raising a Python RuntimeError with the target agent name
- D By restarting the entire workflow from the root agent

Explanation: transfer_to_agent() is the LLM-driven delegation mechanism in ADK. The LLM generates this function call, which ADK's AutoFlow intercepts and routes to the named sub-agent.

Q2
5

What is the purpose of the `output_key` parameter on a Google ADK LlmAgent?

- A To encrypt the agent's response before storing it
- B To automatically save the agent's final response to a named key in shared session.state**
- C To specify which API key the agent should use for LLM calls
- D To define the priority order when multiple agents compete for resources

Explanation: `output_key="result_name"` automatically writes the agent's final text response to `session.state["result_name"]`. The next agent in a SequentialAgent can then read `{result_name}` in its prompt template.

Q2
6

Which ADK mechanism allows a parent LLM agent to call a child agent as an explicitly invocable tool?

- A LoopAgent with shared memory
- B AgentTool — wraps a BaseAgent as a callable tool in the parent's tool list**
- C SessionManager with agent binding
- D DirectCall — a built-in Python function for agent invocation

Explanation: `AgentTool(agent=child_agent)` wraps the child agent and adds it to the parent LlmAgent's `tools=[]`. The parent LLM can then explicitly invoke it as a tool call, controlling exactly when it runs.

Q2
7

A hospital MAS needs a doctor to approve medication orders over 1,000mg before an agent places them. Which ADK pattern handles this?

- A ParallelAgent — the doctor and agent run simultaneously
- B LoopAgent — keep retrying until the doctor is available
- C Human-in-the-Loop via FunctionTool or SecurityPlugin with policy engine**
- D SequentialAgent — the doctor is placed last in the sequence

Explanation: Human-in-the-Loop requires an agent to pause, surface the decision to a human, and resume only after approval. ADK implements this via FunctionTool with an external approval function, or SecurityPlugin with a PolicyEngine.

Q2
8

Which ADK pattern is BEST for improving output quality by having one agent write a draft and another critique it?

- A ParallelAgent with two writers producing competing drafts
- B LoopAgent running the same agent multiple times until output is perfect
- C SequentialAgent([WriterAgent, CriticAgent]) — the Generator-Critic pattern**
- D Human-in-the-Loop where the human acts as the critic

Explanation: The Generator-Critic (or Generator-Reviewer) pattern uses a SequentialAgent: WriterAgent produces a draft (saved to state), then CriticAgent reads it and provides feedback. A LoopAgent can wrap this for iterative refinement.

Q2
9

In Google ADK, how do agents within the same application share data between sequential steps?

A Via REST API calls between agent endpoints

B Through a shared `session.state` dictionary accessible to all agents in the same invocation

C By writing to a shared file on disk and reading from it

D Via a message queue that agents subscribe to independently

Explanation:

`session.state` is the fundamental communication mechanism in ADK. All agents in the same invocation share this key-value store. Agents write with `context.state["key"] = value` and read with `context.state.get("key")`.

Q3
0**FIPA-ACL (Foundation for Intelligent Physical Agents - Agent Communication Language) defines:**

- A A Python library for building AI agents
- B Standard message types called "performatives" that agents use to communicate with defined meanings**
- C A cloud deployment specification for multi-agent systems
- D An encryption protocol for securing agent-to-agent messages

Explanation: FIPA-ACL is an international standard that defines performatives — message types with specific semantic meanings (REQUEST, INFORM, PROPOSE, etc.) that ensure agents understand each other's intent.

Q3
1**An EnergyBudgetAgent wants the HVACControlAgent to reduce zone temperature by 2 degrees. Which FIPA-ACL performative should it use?**

- A INFORM — to tell HVAC the temperature has changed
- B QUERY-IF — to ask if HVAC can reduce temperature
- C REQUEST — to ask HVAC to perform the reduction action**
- D PROPOSE — to suggest the reduction as an option

Explanation: REQUEST is used when Agent A wants Agent B to perform a specific action. INFORM reports facts. QUERY-IF checks conditions. PROPOSE suggests options for negotiation. To command an action, use REQUEST.

Q3
2**In the Contract Net Protocol, which performative does the initiator send to broadcast a task and invite bids from multiple agents?**

- A REQUEST — directly ask each agent to do the task
- B CALL-FOR-PROPOSAL (CFP) — broadcast the task for competitive bidding**
- C AGREE — signal that the task is assigned
- D INFORM — announce what the task is

Explanation: CFP (Call-For-Proposal) is the first step in the Contract Net Protocol. The initiator broadcasts the task; responding agents send PROPOSE with their bid. The initiator then replies ACCEPT-PROPOSAL or REJECT-PROPOSAL.

Q3
3**A WarehouseAgent has successfully fulfilled an order. Which performative should it send to the OrderAgent?**

- A FAILURE — to report the task outcome
- B REFUSE — to confirm it has finished
- C INFORM-DONE — to confirm the action was completed successfully**
- D AGREE — to accept the assignment again

Explanation: INFORM-DONE is used to notify the requesting agent that a previously requested action has been completed successfully. FAILURE is for unsuccessful completion. REFUSE is for declining before starting.

Q3
4

An InventoryAgent wants to know if a specific product is in stock. Which performative should it send to the StockAgent?

- A REQUEST — ask StockAgent to check inventory
- B INFORM — tell StockAgent about the product
- C QUERY-IF — ask whether a specific proposition (product in stock) is true**
- D PROPOSE — suggest what to do if out of stock

Explanation: QUERY-IF is used to ask whether a specific proposition is true (e.g., "is product X in stock?"). It expects an INFORM(true) or INFORM(false) response, not an action.

Q3
5

An "ontology" in MAS communication design refers to:

- A The agent's training dataset and fine-tuning examples
- B The shared vocabulary: objects, actions, attributes, and relations that all agents in the system understand**
- C The execution order in which agents are called
- D The formal contract between the developer and the cloud provider

Explanation: An ontology defines the shared conceptual model of the domain: what objects exist (Room, Order), what actions are possible (set_temperature, ship_item), and how they relate. Without a shared ontology, agents may misunderstand each other.

Q3
6

What is the correct sequence for the basic FIPA-ACL "Request-Reply" protocol?

- A PROPOSE -> ACCEPT -> CFP -> AGREE
- B REQUEST -> AGREE -> (action executes) -> INFORM-DONE or FAILURE**
- C INFORM -> QUERY-IF -> PROPOSE -> REFUSE
- D CFP -> REQUEST -> FAILURE -> AGREE

Explanation: Request-Reply: (1) Agent A sends REQUEST. (2) Agent B replies AGREE (will do it) or REFUSE (will not). (3) If agreed, Agent B executes and replies INFORM-DONE (success) or FAILURE (error).

Q3
7

In hotel energy management ontology design, which best represents the "Actions" component of the ontology?

- A Room, Zone, HVAC, Sensor, GuestProfile
- B temperature, setpoint, zone_id, occupancy_count
- C set_temperature(zone, value), read_occupancy(zone), override_schedule(zone, duration)**
- D Zone has_sensor Sensor, Room belongs_to Zone

Explanation: Actions (verbs) in an ontology define what agents can DO. Objects (nouns) are the entities. Attributes are properties of objects. Relations define how objects relate to each other.

Q3
8

A DeliveryAgent responds to a logistics CFP but cannot meet the deadline. Which performative should it send?

A FAILURE — reporting it cannot complete the delivery

B PROPOSE — with a bid that includes a delay

C REJECT-PROPOSAL — declining the initiator's terms

D REFUSE — declining to make a proposal for the announced task

Explanation:

REFUSE is sent by an agent when it declines to engage with a REQUEST or CFP — it will not make a bid. FAILURE is for after attempting an action. REJECT-PROPOSAL is sent by the INITIATOR when declining a received bid.

Q39 In which collaboration mode do all agents work toward a single shared objective by dividing responsibilities among themselves?

- A Adversarial — agents attempt to detect and counter each other
- B Competitive — agents self-interestedly bid for tasks
- C Cooperative — agents divide labor to achieve a shared goal**
- D Independent — agents pursue separate unrelated goals

Explanation: Cooperative MAS is the most common type in enterprise applications: agents share a goal, divide work by specialization, and coordinate to achieve outcomes no single agent could deliver alone.

Q40 A smart city traffic system assigns road clearing routes to emergency vehicles. Multiple route-planning agents bid based on their estimated clearance time. This is:

- A Cooperative — all agents share the goal of helping emergencies
- B Competitive — route agents bid against each other for the assignment**
- C Adversarial — agents try to block each other's proposed routes
- D Sequential — each agent waits for the previous one to finish

Explanation: Even though the end goal (helping emergencies) is shared, the mechanism (competitive bidding for route assignments) makes this a competitive market-based MAS. Both can coexist in a "mixed" MAS.

Q41 In a "fully observable" environment for a MAS, what is true?

- A Only the orchestrator agent can see all state; sub-agents see partial state
- B Every agent can access all relevant state information at any time**
- C The environment is frozen — it does not change while agents deliberate
- D Agents can observe but cannot affect the environment

Explanation: Full observability means every agent has access to all relevant state information. This simplifies coordination but is rare in real systems. Most real-world environments are partially observable.

Q42 When an agent's environment is partially observable, which technique helps the agent make better decisions despite incomplete information?

- A Increasing the LLM temperature to be more creative
- B Adding more sub-agents until full coverage is achieved
- C Using state estimation — inferring hidden state from observable signals via prediction models or Bayesian methods**
- D Switching to a competitive collaboration mode

Explanation: State estimation compensates for partial observability by maintaining a belief state — a probability distribution over possible true states — updated as new observations arrive. Kalman filters, particle filters, and ML models are common approaches.

Q4
3

Multiple agents in a manufacturing MAS all read from and write to a shared "defect_log" data store. No agent communicates directly with another — they coordinate through the shared store. This architecture is called:

A Request-Reply Protocol

B Contract Net Protocol

C Blackboard Architecture

D Pipeline Architecture

Explanation: Blackboard Architecture uses a shared "blackboard" data store. Agents read problems from the board, post solutions or partial results, and other agents react to these updates. No direct agent-to-agent messaging is needed.

Q4
4

A MAS for supply chain disruption must react to real-time geopolitical news, weather events, and supplier outages — the environment changes faster than agents can fully process it. This is best described as:

A Fully observable and static

B Partially observable and dynamic

C Fully observable and dynamic

D Partially observable and static

Explanation: Real supply chain environments are both partially observable (you can't see all supplier conditions) AND dynamic (disruptions happen continuously). This is the most challenging environment class and requires robust state estimation and rapid response agents.

Q4
5**The Google Agent2Agent (A2A) Protocol is designed to solve which core problem?**

- A Training multiple LLM models on the same dataset simultaneously
- B Enabling agents built on different frameworks (LangGraph, CrewAI, ADK, custom) to communicate and collaborate**
- C Replacing REST APIs with a faster binary protocol for microservices
- D Encrypting agent communication to prevent data leakage

Explanation: A2A solves the interoperability problem: when different teams or vendors build agents on different frameworks, they cannot easily communicate. A2A provides a common protocol so a CrewAI agent can call a LangGraph agent as a peer.

Q4
6**In the A2A Protocol, an "Agent Card" is:**

- A A physical ID card that developers must carry to use the A2A API
- B A JSON descriptor that announces an agent's capabilities, endpoint URL, and authentication method**
- C A credit card connected to the agent's API billing account
- D A dashboard card that displays agent performance metrics

Explanation: An Agent Card is the A2A discovery mechanism: a JSON file at a well-known URL that describes what the agent can do, how to reach it, and how to authenticate. It's how agents find and understand each other.

Q4
7**What is the key conceptual difference between MCP (Model Context Protocol) and A2A Protocol?**

- A MCP is for Python agents; A2A is for TypeScript agents
- B MCP connects agents to tools, APIs, and resources; A2A enables peer-to-peer communication between agents themselves**
- C A2A is deprecated and has been replaced by MCP in all use cases
- D MCP supports real-time streaming; A2A only supports batch processing

Explanation: MCP = how agents access capabilities (tools, databases, APIs). A2A = how agents talk to each other as peers. They are complementary: an agent might use MCP to call a database tool AND A2A to delegate a subtask to another agent.

Q4
8**You have a LangGraph agent that needs to invoke a CrewAI agent as a subcontractor for a specialized task. Which protocol enables this cross-framework delegation?**

- A REST API with a custom JSON payload you design yourself
- B Google A2A Protocol — the standard for cross-framework agent communication**
- C SOAP web services with WSDL definitions
- D GraphQL subscriptions for real-time agent updates

Explanation: A2A is specifically designed for cross-framework, cross-vendor agent interoperability. Without A2A, integrating LangGraph with CrewAI would require custom adapter code. A2A provides the standard interface.

Q4
9

In Google ADK, `session.state` is used for agents within the SAME application. When should you use A2A instead?

- A When agents are defined in the same Python file
- B When two agents use the same LLM model
- C When agents span different services, frameworks, or vendor systems that do not share a codebase**
- D When you want faster communication than `session.state` allows

Explanation: `session.state` is for intra-application agent communication within one ADK invocation. A2A is for inter-application communication: when agents live in separate services, are built by different teams, or use different frameworks entirely.

Q5
0

A production MAS must handle cases where an A2A call to an external agent times out. Which design approach is most appropriate?

- A Crash the entire MAS workflow and require manual restart
- B Ignore the timeout and proceed as if the agent responded successfully
- C Implement fallback logic: retry with exponential backoff, use a cached result, or route to an alternative agent**
- D Switch from A2A to FIPA-ACL for the problematic agent connection

Explanation: Production MAS must be resilient. Best practice for external agent calls: implement retry logic (exponential backoff), use stale cached responses when acceptable, or have a fallback agent that handles the task differently. Never silently ignore failures.